Snake Arcade

Manual Técnico del Proyecto de Laboratorio de Programación III

por Najmah Karyme Zablah
22441365

Tabla de contenido

1. Arquitectura General del Proyecto 4
1.1 Visión de alto nivel 4
1.2 Estructura de archivos 4
1.3 Diagrama de módulos 4
2. Estructuras de Datos (Zerpent.h / Zerpent.cpp) 5
2.1 struct Coordenada 5
2.2 struct Aceituna 5
2.3 struct Estampilla 5
2.4 struct PuntajeRecord 6
2.5 class Serpiente 6
3. Lógica del Juego (Game.h / Game.cpp) 7
3.1 enum class EstadoJuego 7
3.2 enum class Nivel 7
3.3 struct Usuario 7
3.4 class Juego 7
3.4.1 Control de flujo 8
3.4.2 Sistema de colisiones — verificarColisiones() 8
3.4.3 Generación de posiciones 8
3.4.4 Sistema de usuarios y high scores 8
4. Capa de Presentación 9
4.1 PantallaMenu ..... **¡Error! Marcador no definido.**
4.2 PantallaSeleccion ..... **¡Error! Marcador no definido.**
4.3 PantallaJuego ..... **¡Error! Marcador no definido.**
4.3.1 dibujarTablero() 10
4.3.2 dibujarGlow() 10
4.3.3 Overlays ..... **¡Error! Marcador no definido.**

4.3.4 resetearEstadoOverlay() 10

4.4 PantallaScores ..... **¡Error! Marcador no definido.**

4.5 Renderer ..... **¡Error! Marcador no definido.**

5. Flujo del Programa — Main Loop 12

5.1 Punto de entrada (main.cpp) 12

5.2 Diagrama del ciclo principal 12

5.3 Diagrama de transición de estados 13

6. Persistencia de Datos 14

6.1 Formato de usuarios.txt 14

6.2 Formato de highscores.txt 14

6.3 Flujo de lectura / escritura 14

1. Arquitectura General del Proyecto

1.1 Visión de alto nivel

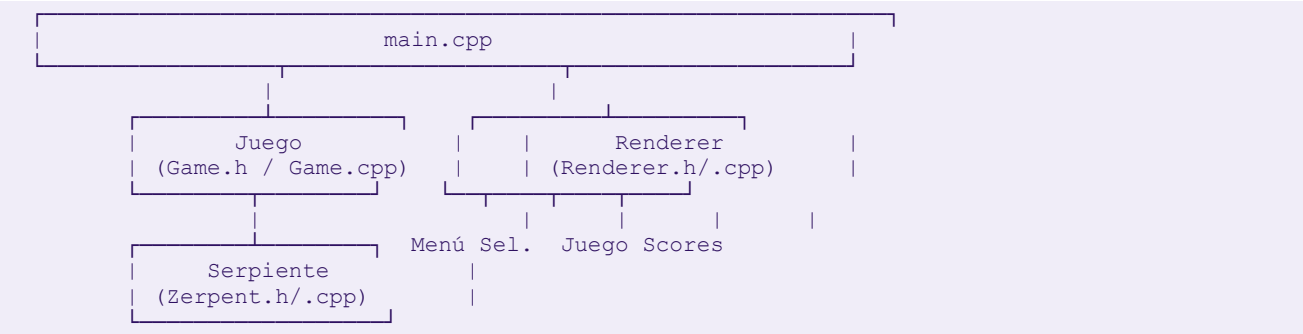
Snake Arcade sigue una arquitectura en capas con separación estricta entre lógica, estado y presentación. La capa de dominio (Zerpent + Game) no depende de SFML en absoluto; la capa de presentación (las cuatro clases Pantalla* más Renderer) consume la capa de dominio a través de una referencia a Juego. El punto de entrada main.cpp solo instancia las dos raíces y ejecuta el bucle.

Capa	Archivos	Responsabilidad
Datos / Entidades	`Zerpent.h / Zerpent.cpp`	Structs: Coordenada, Aceituna, Estampilla, PuntajeRecord. Clase Serpiente.
Lógica del juego	`Game.h / Game.cpp`	Clase Juego: estado, niveles, colisiones, usuarios, high scores.
Presentación	`PantallaMenu / Seleccion / Juego / Scores`	Dibujo y captura de eventos por pantalla.
Coordinación	`Renderer.h / Renderer.cpp`	Delega a la pantalla correcta según EstadoJuego.
Entrada	`main.cpp`	Instancia Juego y Renderer. Ejecuta el game loop.

1.2 Estructura de archivos

```
Proyecto_Snake_Arcade/
├── main.cpp                  Punto de entrada
├── Zerpent.h / .cpp          Entidades y clase Serpiente
├── Game.h / .cpp             Lógica central del juego
├── PantallaMenu.h / .cpp     Login, registro, eliminar cuenta
├── PantallaSeleccion.h/.cpp  Menú principal y selección de nivel
├── PantallaJuego.h / .cpp    Tablero activo, pausa, game over
├── PantallaScores.h / .cpp   Tabla de high scores con tabs
├── Renderer.h / .cpp         Coordinador de pantallas + SFML init
├── Proyecto_Snake_Arcade.pro Qt Creator build file
├── assets/                   PNG: fondos, aceitunas, cajas, estampillas
├── usuarios.txt              Persistencia de cuentas (generado en runtime)
└── highscores.txt            Persistencia de puntajes (generado en runtime)
```

1.3 Diagrama de módulos



2. Estructuras de Datos (Zerpent.h / Zerpent.cpp)

Este par de archivos define todas las entidades del dominio: las cuatro estructuras de datos y la clase Serpiente. No incluye ningún encabezado de SFML, lo que garantiza que la lógica sea independiente de la librería gráfica.

2.1 struct Coordenada

```
struct Coordenada {
    int x, y;
    bool operator==(const Coordenada& otra) const;
};
```

Representa una celda del tablero como un par de enteros. El tablero mide 30 × 28 celdas; cada celda ocupa 20 px en pantalla. El operador == sobrecargado permite comparar dos celdas directamente y es utilizado en las rutinas de colisión.

2.2 struct Aceituna

```
struct Aceituna {
    Coordenada posicion;
    int tipo;      // 0 = verde (+10 pts)  |  1 = dorada (+25 pts)
    int puntos;
    bool activa;
};
```

Coleccionable principal. El campo tipo determina el sprite a renderizar y los puntos otorgados. La aceituna dorada (tipo = 1) aparece solo en el Nivel 3 con probabilidad del 30% y reduce temporalmente la velocidad de la serpiente al factor 0.7 de la velocidad base.

2.3 struct Estampilla

```
struct Estampilla {
    Coordenada posicion;
    int id;          // 0=Mono  1=Erizo  2=Gato
    bool recolectada;
    bool visible;
    int puntosBono;  // siempre 50
};
```

Coleccionable de bonificación, uno por nivel. El campo id se mapea directamente al array de texturas en PantallaJuego. La colisión con la estampilla usa un radio de tolerancia de una celda en ambas dimensiones para compensar su tamaño visual de 54 px.

2.4 struct PuntajeRecord

```
struct PuntajeRecord {
    std::string nombreJugador;
    int puntaje;
    int nivel;      // 1, 2 ó 3
};
```

Unidad de almacenamiento del high score. El vector de estas estructuras se serializa en highscores.txt con el formato nombre|puntaje|nivel, y se ordena por el método Juego::ordenarPuntajes() mediante bubble sort descendente antes de persistirse.

2.5 class Serpiente

Encapsula toda la lógica de movimiento y colisión propia de la serpiente. Internamente mantiene un std::vector<Coordenada> cuerpo donde el índice 0 siempre es la cabeza.

Método	Retorno	Descripción
`Serpiente(x, y, tam=3)`	Constructor	Inicializa el cuerpo horizontal hacia la izquierda.
`mover()`	void	Inserta nueva cabeza (pos + dirección) y elimina la cola si !creciendo.
`cambiarDireccion(dx,dy)`	void	Ignora la dirección opuesta para evitar auto-colisión inmediata.
`crecer()`	void	Activa la bandera creciendo = true; el siguiente mover() omite pop_back().
`getCabeza()`	Coordenada*	Retorna puntero a cuerpo[0]. Permite modificar la cabeza in situ (wrap-around).
`getCuerpo()`	const vector&	Acceso de sólo lectura para renderizar y detectar colisiones externas.
`colisionConSiMisma()`	bool	Compara cuerpo[0] contra cuerpo[1..n] con el operador ==.
`reset(x, y)`	void	Borra el vector y reinicia el cuerpo a 3 segmentos horizontales.
`matar()`	void	Activa viva = false. El estado GAME_OVER es manejado por Juego.

> *¡Ojo! getCabeza() retorna Coordenada* (puntero) y no Coordenada& (referencia) de forma intencional para cumplir el requisito académico de uso explícito de punteros.*

3. Lógica del Juego (Game.h / Game.cpp)

Game.h define los enums de estado y nivel, el struct Usuario y la clase Juego. No contiene ninguna llamada a SFML; toda su dependencia gráfica se limita a los tipos definidos en Zerpent.h.

3.1 enum class EstadoJuego

```
enum class EstadoJuego {
    MENU, CREAR_CUENTA, SELECCION, NIVELES,
    CONFIRMAR_ELIMINAR, JUGANDO, PAUSANDO,
    GAME_OVER, HIGH_SCORES
};
```

Controla qué pantalla dibuja Renderer y qué eventos procesa. Es el pivote central de toda la máquina de estados. El uso de enum class impide colisiones de nombre y prohíbe comparaciones implícitas con enteros.

3.2 enum class Nivel

```
enum class Nivel { UNO = 1, DOS = 2, TRES = 3 };
```

Los valores enteros se usan directamente al serializar el nivel en highscores.txt, evitando una conversión adicional.

3.3 struct Usuario

```
struct Usuario { std::string nombre, contrasena; };
```

Contenedor temporal de credenciales. Solo existe en memoria durante la sesión activa; la persistencia se maneja en usuarios.txt.

3.4 class Juego

Clase central del dominio. Posee todas las entidades del juego y expone una API limpia hacia las clases de presentación. Sus responsabilidades se agrupan en cuatro bloques:

3.4.1 Control de flujo

Método	Descripción
`iniciar(nivel)`	Reinicia puntaje, serpiente, obstáculos y velocidad; genera los primeros coleccionables.
`actualizar()`	Llama a serpiente.mover() y verificarColisiones(). Solo actúa en estado JUGANDO.
`procesarEntrada(dx,dy)`	Delega a serpiente.cambiarDireccion(). Solo actúa en estado JUGANDO.
`pausar()`	Toggle entre JUGANDO ⇄ PAUSANDO.
`setEstado(estado)`	Transición directa de estado (usada por las pantallas de presentación).

3.4.2 Sistema de colisiones — verificarColisiones()

Se ejecuta en cada tick de juego. Obtiene un puntero a la cabeza vía getCabeza() y aplica las siguientes verificaciones en orden:

```
getCabeza() → Coordenada* cabeza
      |
      ├── Nivel 1: wrap-around (aritmética modular sobre cabeza->x, cabeza->y)
      |
      ├── Niveles 2-3: ¿sale del tablero? → GAME_OVER
      |
      ├── colisionConSiMisma() → GAME_OVER
      |
      ├── Nivel 3: ¿coincide con algún obstáculo? → GAME_OVER
      |
      ├── aceituna.activa && *cabeza == aceituna.posicion
      |       └── serpiente.crecer(), puntaje += puntos, generarAceituna()
      |
      └── estampilla visible && |dx|<=1 && |dy|<=1
              └── puntaje += 50, estampilla.recolectada = true
```

3.4.3 Generación de posiciones

Los tres métodos privados generarAceituna(), generarEstampilla() y generarObstaculos() usan rand() % dimensión dentro de un bucle do-while que llama a posicionLibre(). Esta función verifica que la celda candidata no esté ocupada por el cuerpo de la serpiente, los obstáculos existentes ni los coleccionables activos, garantizando cero superposiciones.

3.4.4 Sistema de usuarios y high scores

La persistencia usa std::ifstream / std::ofstream con el delimitador '|'. El formato es texto plano para facilitar la depuración. El método guardarPuntaje() sólo actualiza el récord si el nuevo puntaje supera al almacenado para ese jugador y nivel; después llama a ordenarPuntajes() e imprime los top 10 en el archivo.

```
// Bubble sort descendente (ordenarPuntajes)
for (int i = 0; i < n-1; i++)
  for (int j = 0; j < n-i-1; j++)
    if (lista[j].puntaje < lista[j+1].puntaje)
      swap(lista[j], lista[j+1]);   // intercambio manual (sin std::swap)
```

4. Capa de Presentación

Cada clase de pantalla recibe referencias a los recursos SFML que necesita (ventana, fuente, texturas) por el constructor de Renderer, quien los posee en memoria durante toda la vida del programa. Todas las clases Pantalla* exponen únicamente los métodos públicos dibujar(Juego&) y procesarEventos(sf::Event&, Juego&).

4.1 PantallaMenu

Maneja tres sub-estados: MENU (login), CREAR_CUENTA (registro) y CONFIRMAR_ELIMINAR. Mantiene estado de formulario propio: inputNombre, inputContrasena, inputConfirmar y mensajeError.

Método privado	Descripción
`dibujarLogin()`	Fondo derecho + título centrado en zona derecha con fórmula x = 450 + (350 − w)/2. Dos campos de texto y dos botones.
`dibujarRegistro()`	Tres campos (nombre, contraseña, confirmar). Valida que las contraseñas coincidan antes de llamar a juego.crearCuenta().
`dibujarConfirmarEliminar()`	Solicita la contraseña antes de llamar a juego.eliminarCuenta(). Vuelve a MENU si tiene éxito.
`dibujarCampoTexto()`	Renderiza etiqueta + caja de entrada. El parámetro esClave reemplaza los caracteres visibles por '*'.
`limpiarCampos()`	Resetea todos los string de entrada y mensajeError. Se llama al cambiar de sub-estado.

4.2 PantallaSeleccion

Maneja dos sub-estados: SELECCION (menú de usuario) y NIVELES (elección de dificultad). Además controla el overlay "Ver Estampillas" mediante la bandera booleana mostrarEstampillas.

Método privado	Descripción
`dibujarSeleccion()`	Saludo personalizado con getUsuarioActual(). Cuatro botones: Niveles, Puntajes, Eliminar Cuenta, Cerrar Sesión.
`dibujarNiveles()`	Tres botones de nivel con descripción debajo. Botón "Ver Estampillas" que activa el overlay.
`dibujarOverlayEstampillas()`	Panel oscuro semi-transparente con los tres sprites a 90 px, nombre y nivel de cada estampilla.

4.3 PantallaJuego

La pantalla más compleja. Maneja los estados JUGANDO, PAUSANDO y GAME_OVER y coordina tres overlays: pausa, game over y notificación de estampilla recolectada. Posee su propio timer de animación (timerOverlayEstampilla) actualizado desde Renderer.

4.3.1 dibujarTablero()

Renderiza en este orden: marco + grilla, cajas de obstáculo (PNG 44 px), aceitunas (PNG 44 px con glow), estampilla (PNG 54 px con glow blanco), y por último la serpiente segmento a segmento.

La cabeza (i == 0) recibe tratamiento especial: se deduce la dirección comparando cuerpo[0] con cuerpo[1] y normalizando el resultado para manejar el wrap-around del Nivel 1. Se dibujan dos ojos (blanco + pupila + reflejo) y un sombrero de copa (ala + cuerpo + cinta morada).

4.3.2 dibujarGlow()

```
// N círculos concéntricos con alfa decreciente
for (int i = capas; i >= 1; i--) {
    float r = radio * (i / capas);
    sf::Uint8 a = color.a * (1 - i/(capas+1));
    CircleShape c(r);  c.setFillColor(...a...);
    ventana.draw(c);
}
```

4.3.3 Overlays

Overlay	Comportamiento
Pausa	Panel morado oscuro centrado. Botones Reanudar / Salir. Activado con tecla P o click en botón.
Game Over	Guarda el puntaje una única vez (flag puntajeGuardado). Muestra puntaje y usuario. Botones Reiniciar / Volver.
Estampilla	Panel con fade-in/out (2.8 s). Alpha interpolado en tramos 0–20% (entrada) y 85–100% (salida). Solo visible en estado JUGANDO.

4.3.4 resetearEstadoOverlay()

Se llama cada vez que la pantalla de juego cede el control (Escape o Volver). Reinicia puntajeGuardado, mostrarOverlayEstampilla y estampilaYaMostrada para que el siguiente nivel empiece limpio.

4.4 PantallaScores

Carga la lista completa con juego.cargarPuntajes() (ya ordenada por bubble sort) y la filtra por el nivel de la tab activa. Muestra máximo 10 registros con colores diferenciados para el top 3 (dorado, plateado, bronce).

Tab	Descripción	Filtro
Nivel 1 — Fácil	Fondo verde oscuro. Tab activa con borde dorado y subrayado.	`nivel == 1`
Nivel 2 — Medio	Fondo amarillo oscuro.	`nivel == 2`
Nivel 3 — Difícil	Fondo rojo oscuro.	`nivel == 3`

4.5 Renderer

Posee todos los recursos SFML (ventana, fuente, 9 texturas) y las cuatro instancias de pantalla. Es el único punto de acoplamiento con SFML en el nivel de coordión. Sus tres métodos públicos mapean directamente a las tres fases del game loop.

Método público	Descripción
`procesarEventos(juego)`	Itera pollEvent(). Delega al objeto Pantalla* correcto según EstadoJuego.
`actualizar(dt, juego)`	Solo delega a pantallaJuego.actualizar(dt) si el estado es JUGANDO, PAUSANDO o GAME_OVER.
`dibujar(juego)`	ventana.clear() → delega a la pantalla correcta → ventana.display().
`cargarTexturas()`	Carga los 9 PNG desde la carpeta assets/. Imprime en cerr si alguno falla (no lanza excepción).

5. Flujo del Programa — Main Loop

5.1 Punto de entrada (main.cpp)

```
int main() {
    Juego    juego(30, 28);      // tablero 30x28 celdas
    Renderer renderer(800, 600); // ventana 800x600 px

    sf::Clock reloj;      // controla la velocidad de la serpiente
    sf::Clock relojDelta; // controla dt de animaciones

    while (renderer.ventanaAbierta()) {
        float dt = relojDelta.restart().asSeconds();

        renderer.procesarEventos(juego);   // 1. Eventos

        if (juego.getEstado() == EstadoJuego::JUGANDO)
            if (reloj.getElapsedTime().asSeconds() >= juego.getVelocidad()) {
                juego.actualizar();          // 2. Lógica
                reloj.restart();
            }

        renderer.actualizar(dt, juego);    // 3. Timers visuales
        renderer.dibujar(juego);            // 4. Render
    }
    return 0;
}
```

El juego usa dos relojes independientes: reloj controla cada cuántos segundos avanza la serpiente (getVelocidad() devuelve entre 0.10 y 0.25 s según el nivel y el puntaje), mientras relojDelta proporciona el delta-time en segundos para los timers de animación como el fade del overlay de estampilla.

5.2 Diagrama del ciclo principal

```
┌──────────────────────────────────────────┐
│              INICIO                      │
│   Juego(30,28) + Renderer(800,600)       │
└──────────────────┬───────────────────────┘
                   │
┌──────────────────┴──────────────┐  ventana
│   ▼ ventana abierta?  ──────────┐  cerrada
└─────────────────────────────────┘  │ → return 0
    │                                 │
    ▼ SÍ                              │
┌────────────────────┐              │
│ dt = relojDelta.restart() │        │
└────────┬───────────┘              │
         │                          │
    ▼ FASE 1: procesarEventos()     │
    ▼ FASE 2: si JUGANDO && tick    │
              juego.actualizar()    │
    ▼ FASE 3: renderer.actualizar(dt) │
    ▼ FASE 4: renderer.dibujar()    │
        │                           │
        └───────────────────────────┘
```

5.3 Diagrama de transición de estados

```
MENU ──────────────────── [Crear Cuenta] ──────── CREAR_CUENTA
 |  ←──────────────────────────────────────────── [Volver/Éxito]
 |
[Login OK]
 |
SELECCION ──────────────── [Puntajes] ──────── HIGH_SCORES
 |     │  ←──────────────────────────── [Volver]
 |     └──────── [Eliminar] ─── CONFIRMAR_ELIMINAR ── MENU
 |
[Niveles]
 |
NIVELES ──────────────── [Nivel 1/2/3] ──── JUGANDO
 ↑                                          |     |
 └──────────────────── [Esc/Volver] ───┘     |
                                             |
                              [P] ── PAUSANDO ── [P]
                                             |
                            [muerte] — GAME_OVER — [R] → JUGANDO
```

6. Persistencia de Datos

6.1 Formato de usuarios.txt

```
# Una línea por cuenta
nombre_usuario|contraseña

# Ejemplo:
Najmah|abc123
juan_42|password
```

Cada línea se parsea localizando el primer '|' con std::string::find(). Si no se encuentra (línea malformada), se omite con continue. La escritura sobrescribe el archivo completo con std::ofstream en cada modificación.

6.2 Formato de highscores.txt

```
# Una línea por record (top 10 global, ya ordenado)
nombre_jugador|puntaje|nivel

# Ejemplo:
Najmah|210|1
juan_42|185|2
Najmah|95|3
```

Se guarda un solo récord por jugador por nivel. Si el jugador ya tiene un récord en ese nivel, se actualiza solo si el nuevo puntaje es mayor. El archivo siempre contiene máximo 10 líneas tras el ordenamiento.

6.3 Flujo de lectura / escritura

```
ESCRITURA (guardarPuntaje)
──────────────────────────────────────────────────
cargarPuntajes() → vector<PuntajeRecord>
    ▶ ¿existe record del jugador en ese nivel?
            SÍ: actualiza si puntaje > anterior
             NO: push_back nuevo record
    ▶ ordenarPuntajes()  [bubble sort desc]
    ▶ ofstream  →  imprime top 10 a highscores.txt

LECTURA (cargarPuntajes)
──────────────────────────────────────────────────
ifstream → getline() por línea
    ▶ find('|') → p1, find('|', p1+1) → p2
    ▶ Si p1 o p2 == npos  →  continue (línea inválida)
    ▶ substr(0, p1)       →  nombreJugador
    ▶ stoi(substr(p1+1))  →  puntaje
    ▶ stoi(substr(p2+1))  →  nivel
    ▶ push_back a vector<PuntajeRecord>
```

> *¡Ojo! Los archivos se crean automáticamente por el sistema operativo al abrir std::ofstream si no existen. No se requiere ninguna inicialización manual.*